Large-Scale Terrain Data Analysis 2: terrain visibility

Mid-submission report. Team: Person 1 (Track 1, radar visibility), Person 2 (Track 2, flight-route visibility), Person 3 (Track 3, optimal visibility corridor).

We build three terrain-visibility applications on one shared set of DEMs and measure how their cost grows with terrain size and problem parameters: runtime, peak memory, scalability, complexity, rendering performance and accuracy. Every number in this report comes from the code and result files in the project folder (*results/*).

Part A (sections 1-5) is the mid-submission scope (about 50% of the project). Part B (sections 6-9) is additional completed work for the final submission.

1. Shared setup (Phase 0)

Data. Copernicus GLO-30 DSM, Garhwal Himalaya (Uttarakhand), reprojected with bilinear resampling to WGS 84 / UTM 44N (EPSG:32644) at 30 m. Four square DEMs share the same centre (Rudraprayag) and the same 30 m cells, so a bigger DEM means more *area*, not finer cells. No no-data cells.

DEM	cells	extent	min (m)	max (m)	relief (m)	std (m)
500 x 500	250,000	15 x 15 km	594	2,856	2,263	448
1000 x 1000	1,000,000	30 x 30 km	547	3,022	2,475	479
2000 x 2000	4,000,000	60 x 60 km	485	4,480	3,995	556
4000 x 4000	16,000,000	120 x 120 km	315	7,074	6,759	1,122

Table 1. The four benchmark DEMs. Larger DEMs reach the high Himalaya (up to 7,074 m), so terrain also gets more rugged with size; the results are read with that in mind.



Figure 1. The 4000 x 4000 DEM (120 x 120 km), hillshade over elevation, rendered in QGIS.

Visibility model (all tracks). A cell is visible if the point *target height* above it is in line of sight from the observer, whose eye is *observer height* above its cell. Earth curvature with standard refraction is applied (drop = (1 - k) d^2 / 2R, k = 1/7). Radars stand on 30 m masts; terrain targets are at 0 m; aircraft targets for Track 3 are 150 m above

ground. Rasters are GeoTIFF (1 visible, 0 blocked, 255 out of range), routes GeoJSON, metrics CSV.

Benchmark protocol. Each measured run is a fresh Python process (*common/bench.py*), repeated 5 times (3 or 1 for the slowest configurations); we report the median and keep every run in the CSV. *compute* = algorithm only; *runtime* also includes reading the DEM; *peak memory* = peak working set of the process (about 60 MB is Python, NumPy and GDAL themselves). Machine: AMD Ryzen 9 7900X (12 cores / 24 threads), 31.1 GB RAM, Windows 11, Python 3.12.13, GDAL 3.13.1, NumPy 2.4.6.

2. Track 1: radar visibility assessment (Person 1)

Design and implementation

Three viewshed algorithms behind one interface (*radar/viewshed.py*), same physical model and output:

- **Naive line of sight**: for every cell, sample the terrain profile to the observer every half cell (bilinear) and test whether anything rises above the sight line. O(N · R) for N cells and R cells of radius. Exact; used as the accuracy reference.
- **R2 ray casting** (Franklin and Ray): one ray to every cell on the boundary square, each ray keeps a running horizon. O(N). Our own implementation, vectorised over rays in NumPy.
- **GDAL viewshed** (C++, Wang et al. reference planes), O(N). The trusted production implementation.

Validation

Test	naive	R2	GDAL
flat plane: all visible	100.0%	100.0%	100.0%
V-shaped valley: all visible	100.0%	100.0%	100.0%
200 m wall: everything behind hidden	100.0%	100.0%	100.0%
real DEM 500, agreement with exact LOS	reference	99.7% (kappa 0.957)	99.8% (kappa 0.965)
real DEM 1000, 10 km range	reference	99.8%	99.8%

Table 2. Correctness on synthetic terrains with known answers and against the exact line-of-sight reference on real terrain. An early R2 version scored 84.7% on the valley because it tested the cell centre against a horizon built from interpolated ray points; testing the ray point itself fixed it.

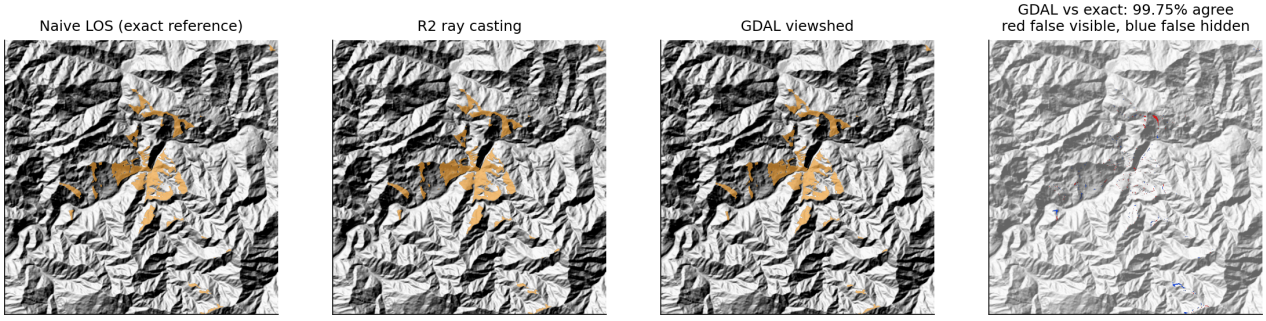


Figure 2. Same radar, three algorithms (DEM 500). Right: where GDAL differs from the exact answer; the few mismatches sit on grazing ridge edges.

Scenario: six radar sites

Six ridge-top sites (highest point within 3 km, 1500-3200 m, at least 22 km apart; *common/sites.py*), range 40 km. Together they see **16.7%** of the ground in the 120 x 120 km area, but would detect an aircraft 150 m above the ground over **35.6%** of it. Individual radars see 3.8% to 14.4% of the ground inside their range: deep valleys hide most terrain.

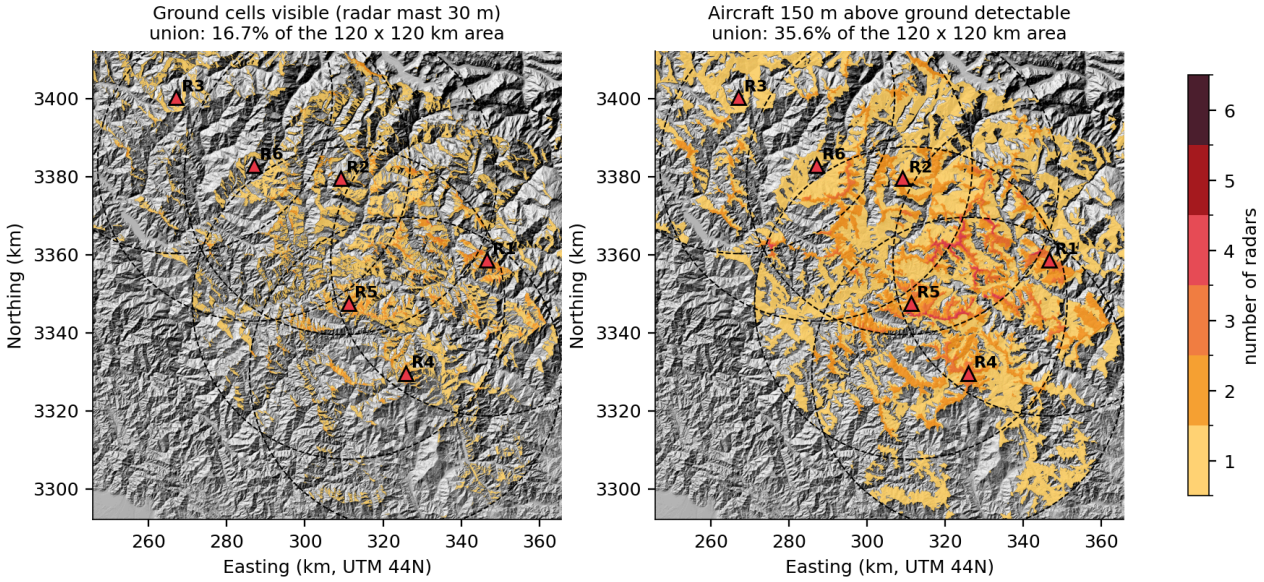


Figure 3. Number of radars that see each ground cell (left) and each aircraft position 150 m above ground (right). Dashed: 40 km range.

First benchmark: runtime and memory vs DEM size (E1)

DEM cells	naive (s)	R2 (s)	GDAL (s)	GDAL peak MB
250,000	6.62	0.027	0.004	68
1,000,000	56.37	0.088	0.010	72
4,000,000	not run	0.326	0.031	103
16,000,000	~4,087 (projected)	1.585	0.100	242

Table 3. Radar at the centre, unlimited range (the whole DEM is processed), median compute time.

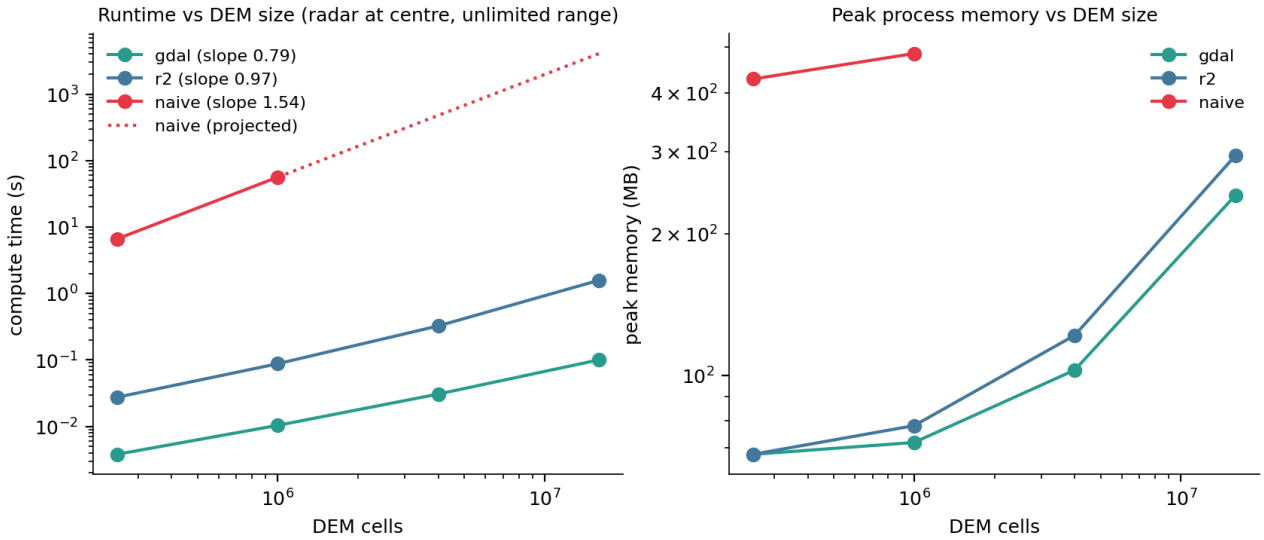


Figure 4. Log-log runtime and memory vs number of cells; the slope is the empirical complexity exponent.

The fitted exponents match the theory: naive LOS grows as $N^{1.54}$ (theory $N^{1.5}$), R2 as $N^{0.97}$ and GDAL as $N^{0.79}$ (theory N; fixed overheads flatten the small sizes). Extrapolated, the exact method would need about **68 minutes** for the 16 M-cell DEM, against 0.10 s for GDAL, while agreeing to within about 0.3%. Memory is dominated by the DEM array and the output (4 + 1 bytes per cell).

3. Track 2: flight-route visibility (Person 2)

Design and implementation

- **Route.** A 166 km valley route along the Alaknanda (Devprayag, Srinagar, Rudraprayag, Karnaprayag, Chamoli, Joshimath), found as the least-elevation path between the towns and simplified to 300 m (*flight/route.py*).
- **Altitude.** Terrain following at 300 m above a smoothed ground (running maximum over 3 km, since an aircraft cannot follow every bump), or a constant cruise altitude. The profile is computed once at 100 m spacing, so every sampling density flies the same path.
- **Visibility.** The route is sampled every 1,000 m (167 positions); at each, a viewshed within a 20 km sensor radius (GDAL engine, same curvature model). Results are aggregated into: how many positions see each cell, the first position that sees it, per-position and newly seen area, and **blind spots** (cells within the radius of the route that are never seen).

Scenario results

Profile	area seen	corridor seen	blind in corridor	mean visible per position
Terrain following, 300 m AGL	1,537 km2	25.5%	4,487 km2	51 km2
Cruise, 5500 m MSL	4,732 km2	78.6%	1,292 km2	698 km2

Table 4. The corridor is the 6,024 km^2 within 20 km of the route.

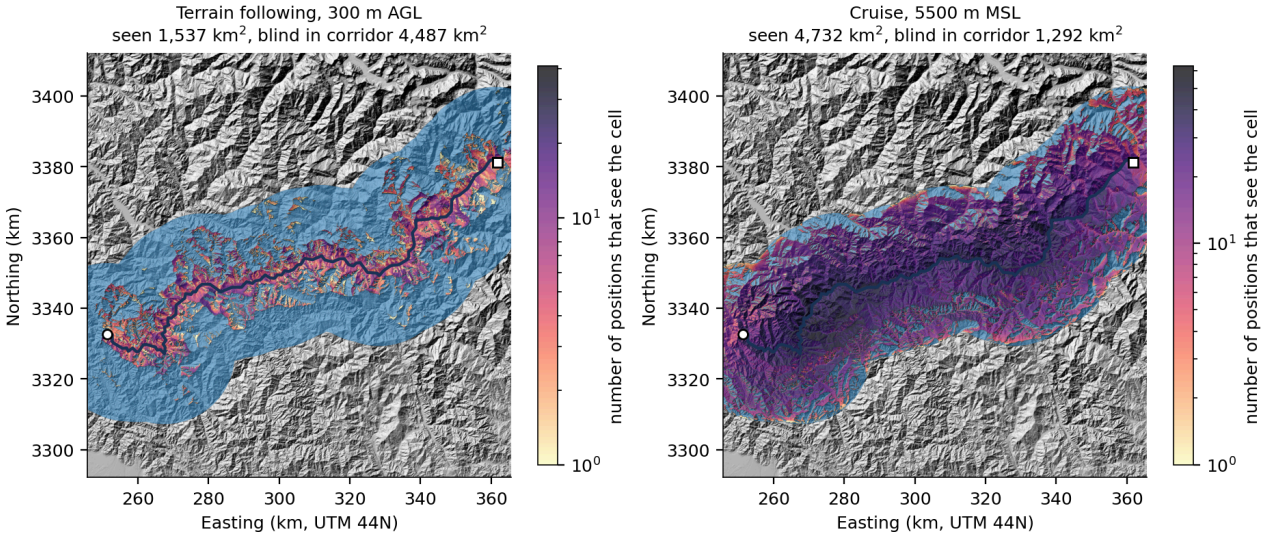


Figure 5. Number of positions that see each cell (colour) and blind spots in the corridor (blue). Low valley flying sees only the valley walls.

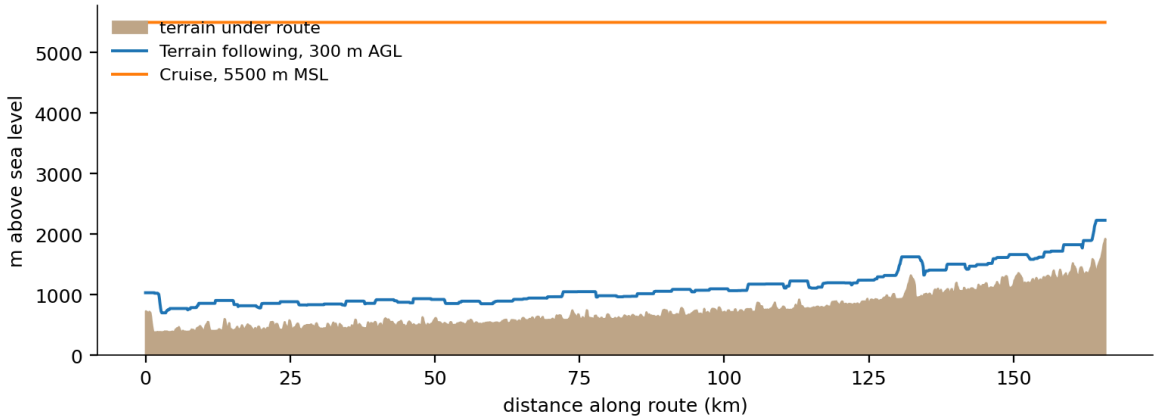


Figure 6. Terrain under the route and the two flight profiles.

Flying low in the valley is the stealthy option: the aircraft sees (and is seen from) only 25.5% of the corridor; at 5,500 m it sees 78.6%. Each scenario takes about 4.6 s for 167 viewsheds.

First benchmark: runtime vs DEM size (F1)

DEM cells	F1: route = 0.8 x DEM width, unlimited radius (s)	positions	F1b: fixed 12 km route, 20 km radius (s)
250,000	0.07	13	0.07
1,000,000	0.43	25	0.21
4,000,000	2.81	49	0.31
16,000,000	18.78	97	0.31

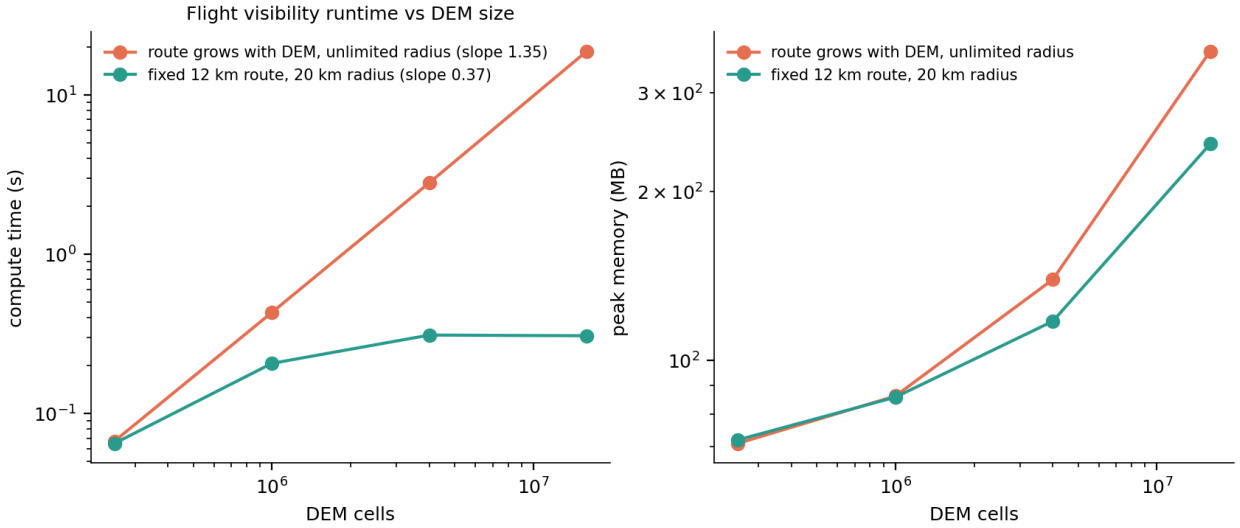


Figure 7. Two ways to grow the problem with DEM size.

When the route and the sensor radius grow with the terrain (F1), each viewshed covers the whole DEM and the number of positions grows with its width, so cost grows like N · √N: measured exponent 1.35. With a fixed route and radius (F1b) only reading the larger DEM grows, exponent 0.37. The work is set by *positions x cells per viewshed*, not by DEM size itself.

4. Track 3: optimal visibility corridor (Person 3)

Objective and design

Visibility meaning. Track 3 uses *visibility of the aircraft from ground radars*: find a route between two points that keeps the aircraft hidden from the six Track 1 radars without making it much longer. The aircraft flies 150 m above the ground; terrain where that is above a 5,000 m ceiling is blocked.

Exposure E(cell) = sum over radars that can see an aircraft there of (1 - d / range), range 40 km (closer radars count more). **Edge cost** between neighbouring cells u, v on the 8-connected grid:

$$c(u, v) = L_{uv} (1 + \alpha (E_u + E_v)/2) + \beta \max(0, z_v - z_u)$$

L = 30 or 42.4 m; α trades exposure against distance; β = 2 penalises climbing. α = β = 0 is the shortest-distance baseline.

Solvers (*corridor/optimizer.py*): Dijkstra in pure Python with a binary heap (baseline); A* with h(u) = straight-line distance + $\beta \max(0, z_{goal} - z_u)$, admissible and consistent because every metre costs at least 1 and any path must climb at least the height difference; SciPy's C Dijkstra on an explicit sparse graph; and a coarse-to-fine multi-resolution search (approximate). All exact solvers return identical optimal costs (checked on every size).

Scenario: Devprayag to Joshimath

Route	length	exposure (weighted km)	length exposed	climb
shortest distance (alpha 0)	130.6 km	31.5	39.4%	26,492 m
least exposure (alpha 10, beta 2)	170.0 km	2.2	2.8%	21,639 m
Track 2 valley route	177.9 km	26.5	25.9%	19,408 m

Table 5. Same endpoints as the Track 2 route. Exposure integral = sum of segment length x exposure.

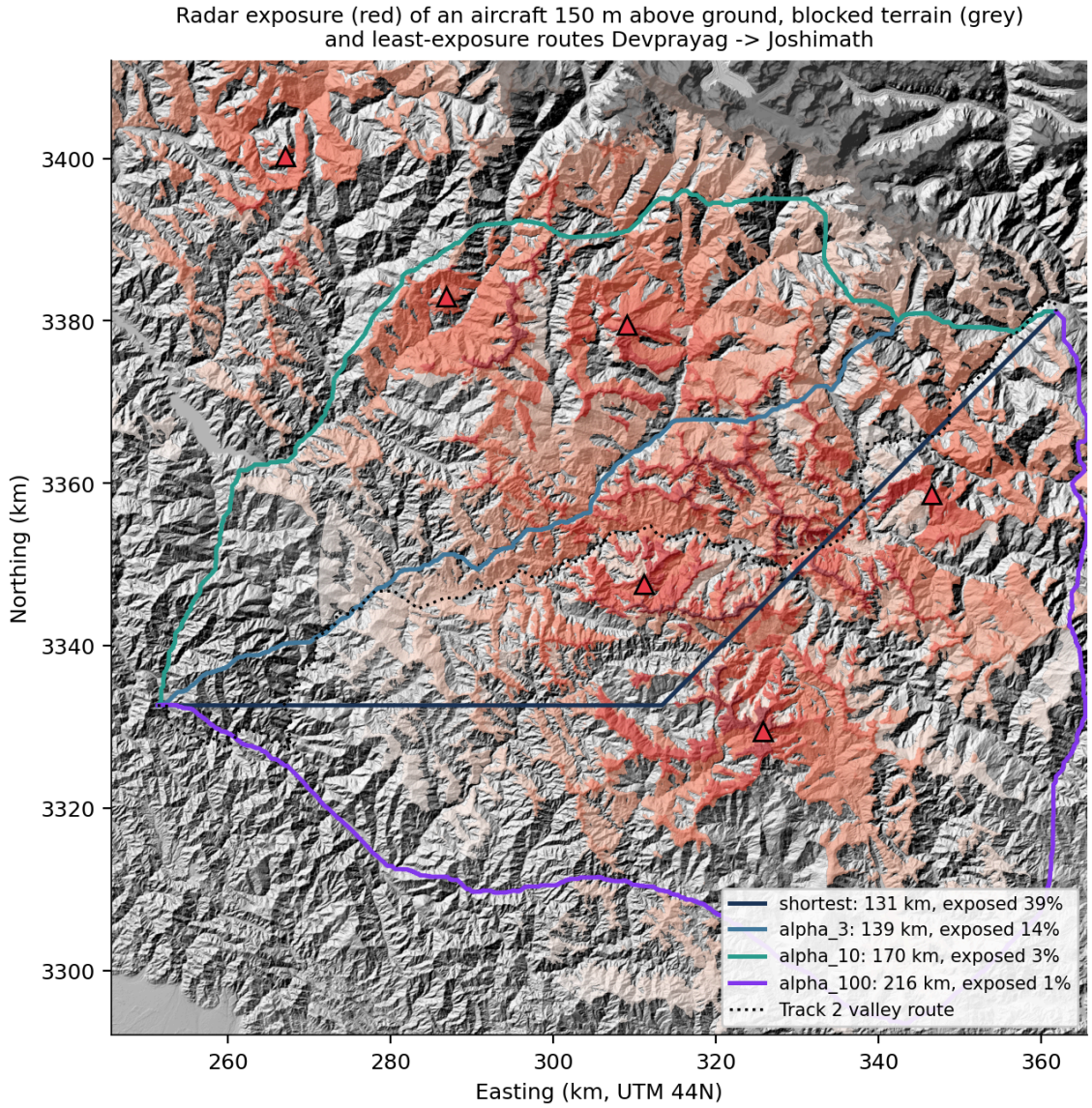


Figure 8. Exposure surface, blocked terrain and routes for several alpha.

The least-exposure route is 30.2% longer than the shortest one but cuts the exposure integral by 93.1%.

First benchmark: search time vs graph size (C1)

nodes	Dijkstra (Python) s	A* (Python) s	SciPy C Dijkstra s	multi-res s	multi-res cost gap
250,000	0.73	0.79	0.11	0.10	0.02%
1,000,000	2.76	2.92	0.48	0.27	0.00%
4,000,000	16.07	15.09	1.97	0.93	0.08%
16,000,000	70.24	54.96	8.98	3.04	1.40%

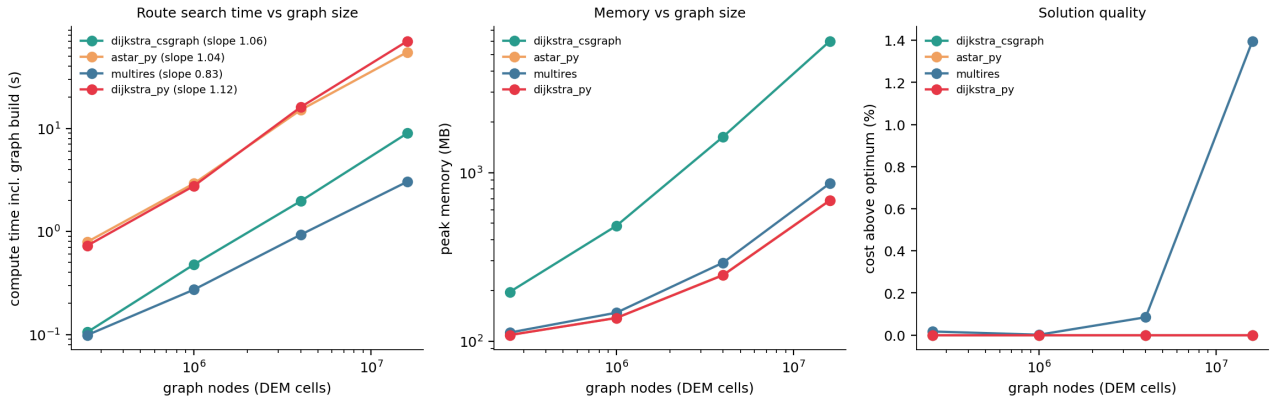


Figure 9. Search time (incl. graph build), memory and cost above the optimum vs graph size.

Exponents: Python Dijkstra 1.12, A* 1.04, SciPy 1.06 (theory O(N log N) for an N-node grid). Moving the same algorithm from Python to C gains about 8x at 16 M nodes, but the explicit graph costs memory (6,025 MB vs 683 MB).

5. Status and plan

All three applications run end to end, pass their correctness checks, export GeoTIFF/GeoJSON/CSV, and have a first scaling benchmark. A styled QGIS project (*qgis/terrain_visibility.qgz*) holds every output. Remaining phases for the final submission: parameter studies, optimisation comparisons, rendering performance and the cross-track analysis. Part B shows that these have already been run.

6. Part B. Track 1: parameters and optimisation

Part B: completed work beyond the 50% scope.

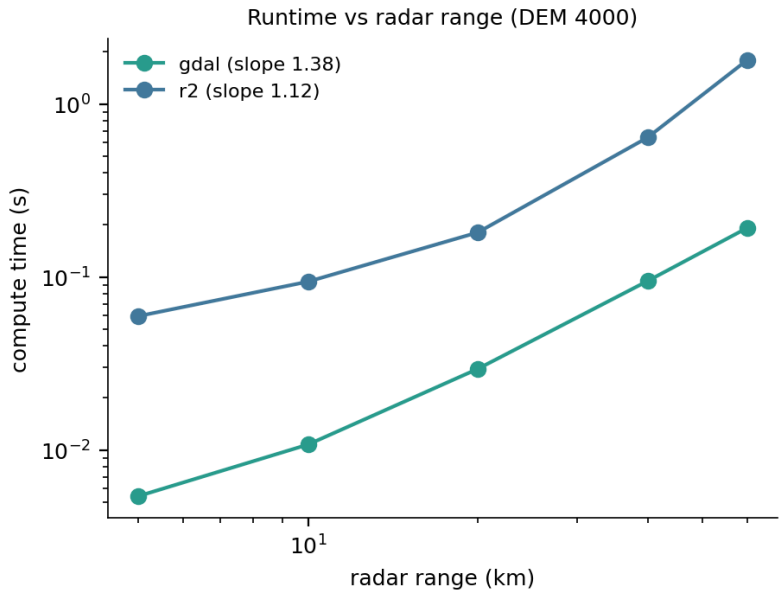


Figure 10. Runtime vs radar range on the 4000 DEM.

Range (E2). Work grows with the area inside the range: compute time slope 1.38 (GDAL) and 1.12 (R2) against range (theory 2 for the area; GDAL's fixed costs dominate at short range). **Height (E3)** changes coverage, not cost.

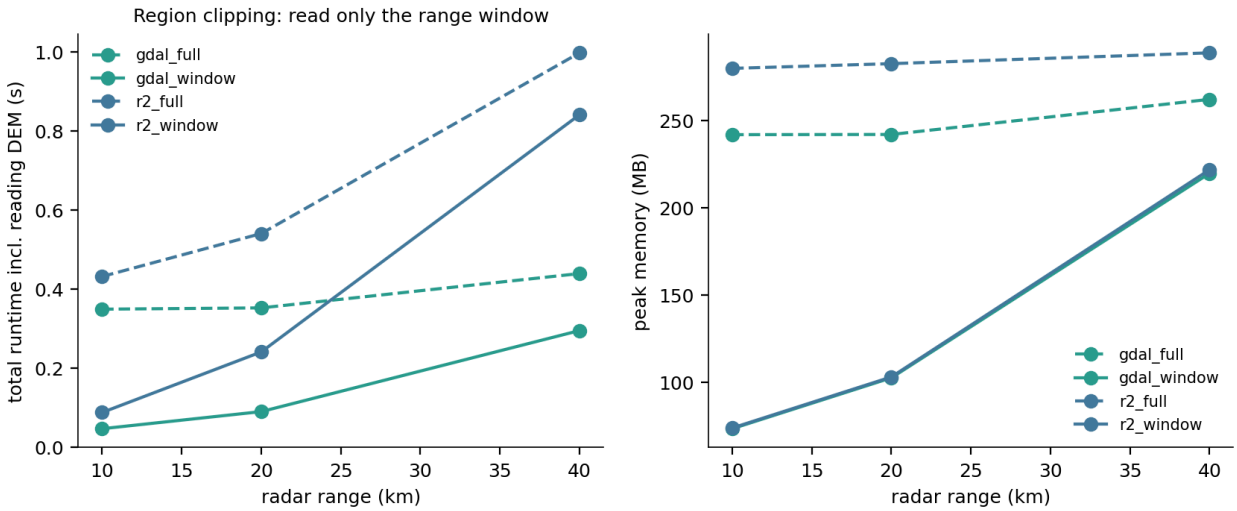


Figure 11. Region clipping: reading only the window that the range covers.

range	full read: runtime / peak	window read: runtime / peak
10 km	0.348 s / 242 MB	0.046 s / 73 MB
20 km	0.352 s / 242 MB	0.089 s / 103 MB
40 km	0.439 s / 262 MB	0.294 s / 220 MB

Table 6. GDAL with and without region clipping (E5), DEM 4000.

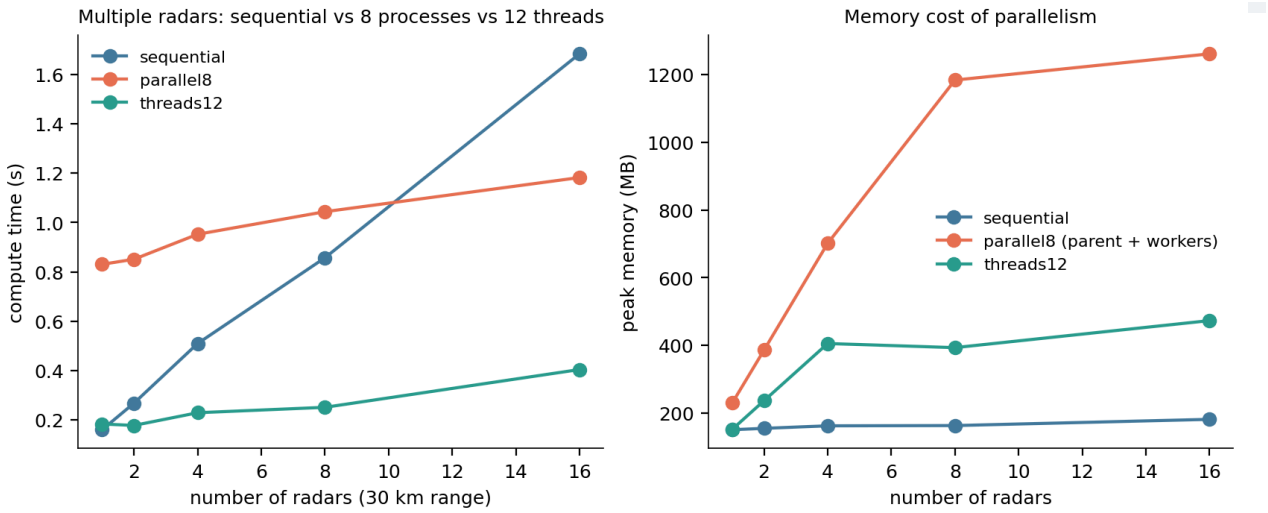


Figure 12. Several radars: sequential vs 8 worker processes (E4).

Parallel radars (E4). The first attempt used 8 worker processes: on Windows each starts from scratch (about 0.8 s, longer than one 30 km viewshed) and results are copied back, so it only won from 16 radars (1.42x). GDAL releases Python's global lock inside the viewshed, so 12 threads in one process run truly in parallel: 1.51x at 2 radars and 4.16x at 16 (1.68 s to 0.40 s), at about 0.5 GB because the windows are held together.

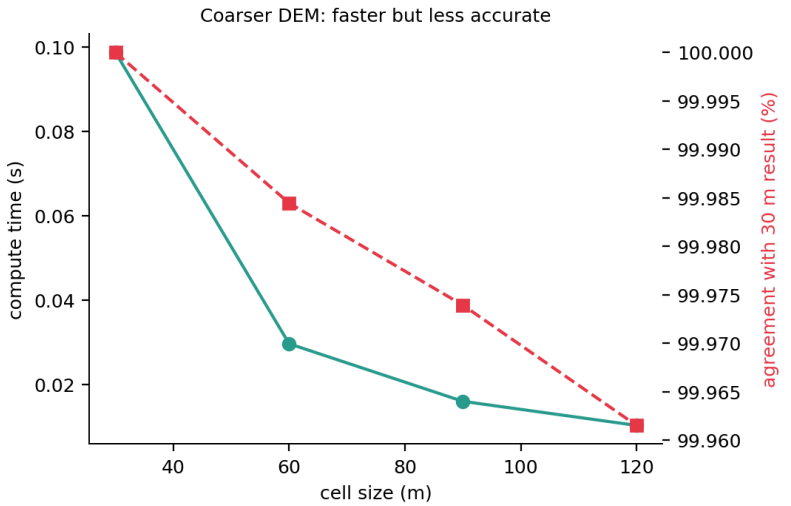


Figure 13. Coarser DEM: faster but less accurate (E6).

cell size	compute	agreement	kappa
30 m	0.099 s	100.00%	1.000
60 m	0.030 s	99.98%	0.898
90 m	0.016 s	99.97%	0.835
120 m	0.010 s	99.96%	0.777

Table 7. Agreement with the 30 m result. Raw agreement stays high because most cells are hidden either way; kappa (chance-corrected) shows the real loss.

7. Part B. Track 2: parameters and optimisation

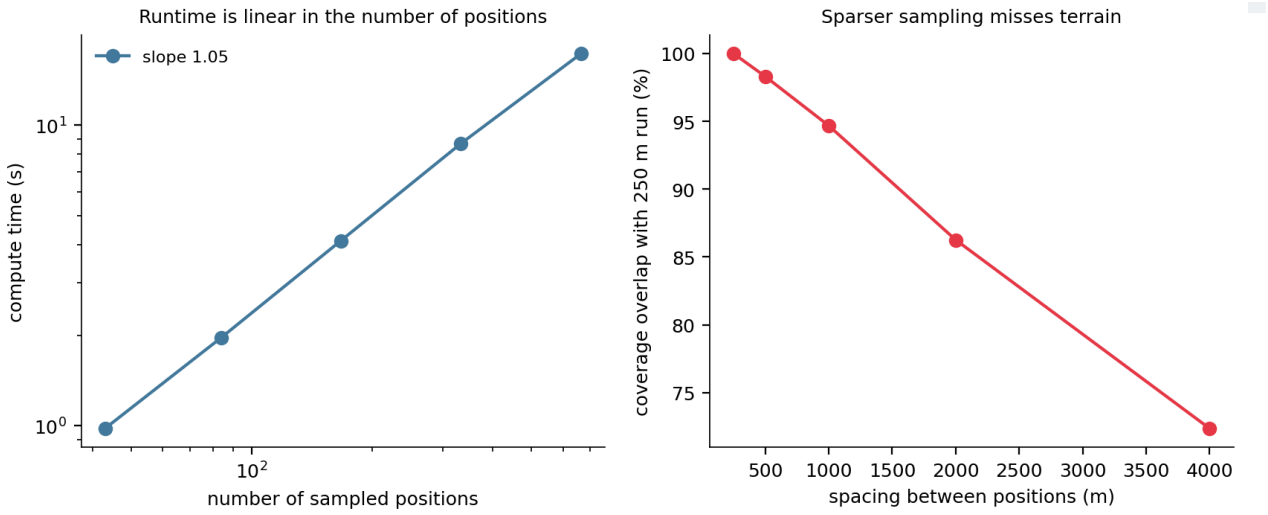


Figure 14. Number of positions vs time, and coverage lost by sparser sampling (F2).

spacing	positions	compute	overlap with 250 m run	missed area
250 m	665	17.31 s	100.0%	0 km2
500 m	333	8.69 s	98.3%	27 km2
1000 m	167	4.13 s	94.7%	86 km2
2000 m	84	1.97 s	86.3%	223 km2
4000 m	43	0.98 s	72.3%	449 km2

Table 8. Runtime is linear in the number of positions (slope 1.05).

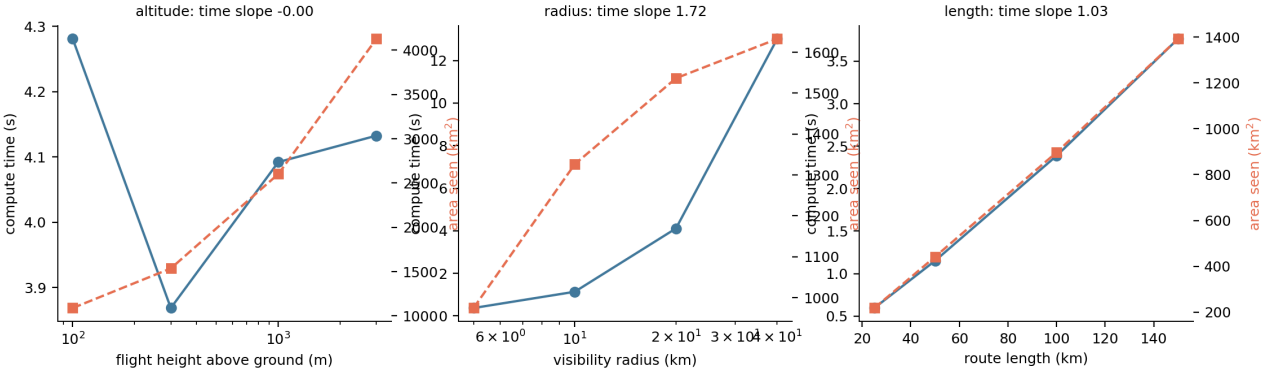


Figure 15. Flight height, sensor radius and route length (F3-F5).

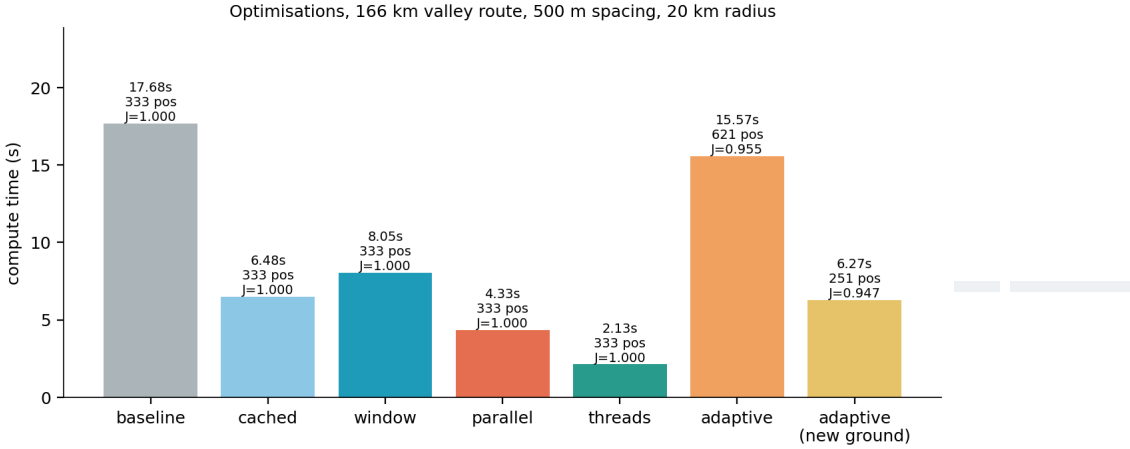


Figure 16. Optimisations on the 166 km route, 500 m spacing (F6).

mode	compute	positions	peak memory	same coverage as dense (Jaccard)
baseline	17.68 s	333	244 MB	1.000
cached	6.48 s	333	309 MB	1.000
window	8.05 s	333	243 MB	1.000
parallel	4.33 s	333	750 MB	1.000
threads	2.13 s	333	684 MB	1.000
adaptive	15.57 s	621	1,210 MB	0.955
adaptive_gain	6.27 s	251	615 MB	0.947

Table 9. Optimisations (F6). Baseline re-opens the DEM file for every position.

Findings. Runtime is linear in the number of positions and route length, roughly quadratic in the sensor radius (slope 1.72) and independent of flight height (slope -0.00), while the area seen rises with both. Keeping the DEM in memory gives 2.7x; copying a range window per position adds overhead because GDAL already limits itself to the radius. Worker processes give 4.1x, but **threads** (GDAL releases the global lock) give 8.3x with identical results. Adaptive sampling did not pay off. The first criterion (refine where neighbouring viewsheds differ) refines almost everywhere in a winding valley (621 positions). The second (refine only where the midpoint sees new ground) uses 251 positions at Jaccard 0.947, which is no better than fixed spacing (Figure 16b): coverage grows smoothly along the route, so uniform spacing is already efficient. The useful lever is to match the spacing to the altitude: at cruise, 4 km spacing keeps 97.5% of the coverage with 43 viewsheds; valley flying needs 500 m.

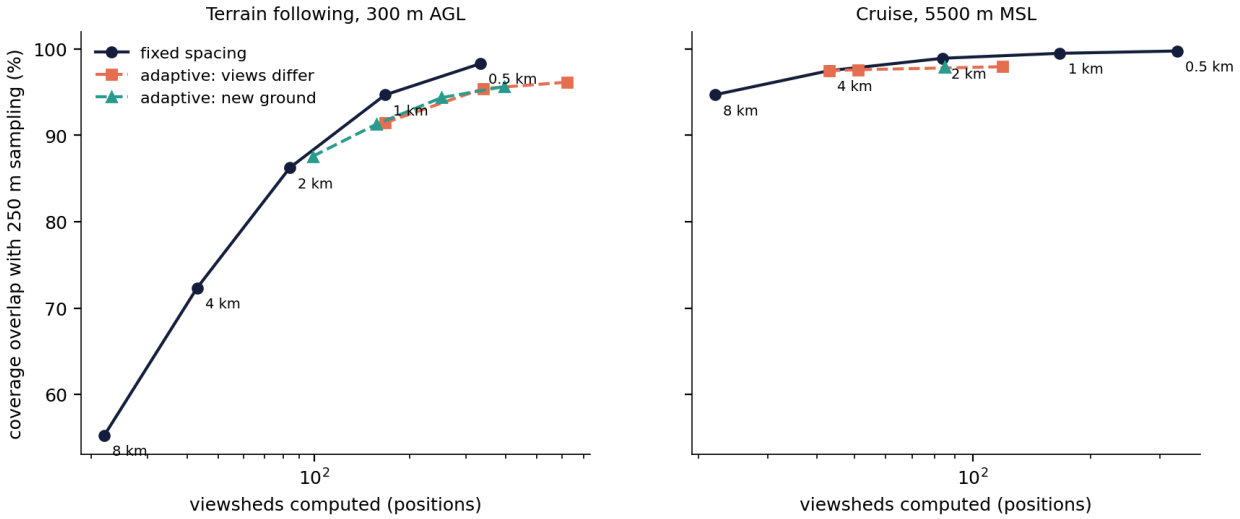


Figure 16b. Positions vs coverage for fixed spacing and both adaptive criteria, valley flying and cruise.

8. Part B. Track 3: parameters and optimisation

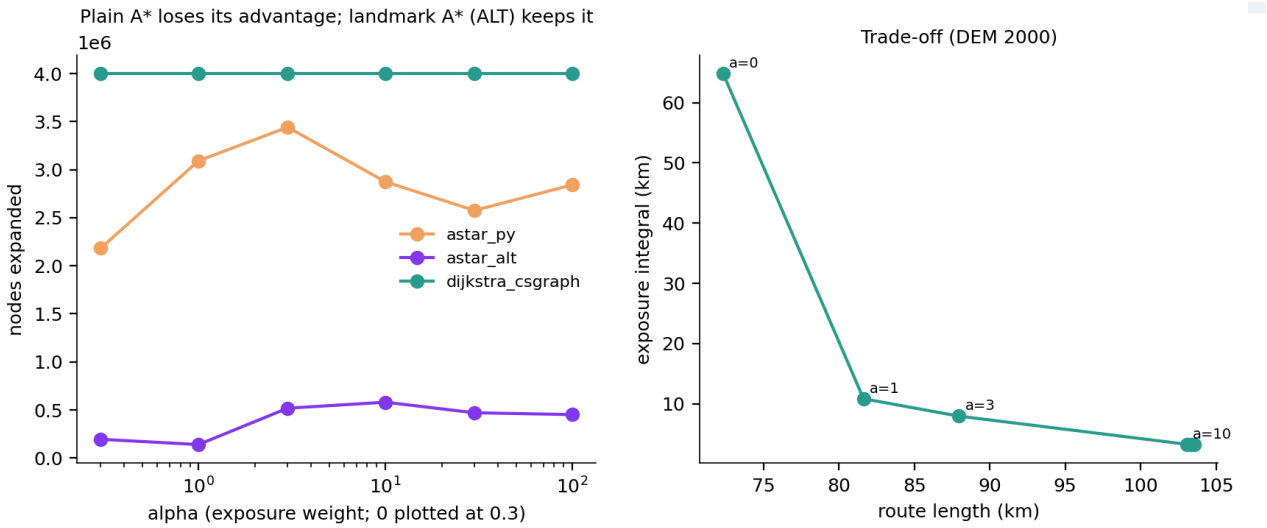


Figure 17. Effect of the exposure weight alpha (DEM 2000): nodes expanded and the length vs exposure trade-off.

Exposure weight (C2) and the A* fix. Plain A* uses straight-line distance as its estimate of the remaining cost, assuming 1 cost unit per metre, but exposed metres cost 5-17 units. With alpha 1 to 100 it therefore expands 64.4% to 86.0% of the 4 M nodes. **Landmark A* (ALT)** fixes this: exact costs to and from 4 landmarks are precomputed once (one forward and one backward C Dijkstra each), and by the triangle inequality give a much tighter estimate that is still admissible and consistent. It returns the same optimum while expanding only 3.5% to 14.5% of the nodes at every alpha, and answers in 0.75-1.95 s instead of 8.1-12.3 s. On the 16 M-node scenario: 4,555,338 nodes and 21.5 s instead of 14,335,945 nodes and 67.3 s, after 36.9 s and 488 MB of one-off preprocessing. The trade-off curve flattens after alpha = 10: the route length goes from 72.3 to 103.1 km while exposure falls from 64.8 to 3.2.

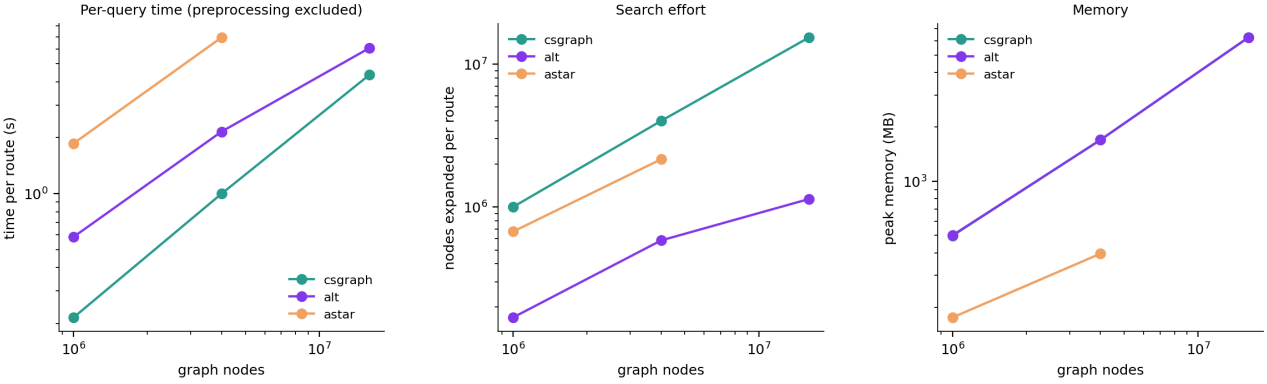


Figure 17b. Eight routes on the same map (C6): time per route, nodes expanded and memory.

Many routes on one map (C6). Per route, landmark A* is about 3x faster than plain A*, but in Python it does not beat the C Dijkstra (6.08 s vs 4.36 s at 16 M nodes) even though it expands only 7.1% of the nodes: Python costs far more per node. Implementing it in compiled code is the next step. All methods return identical route costs.

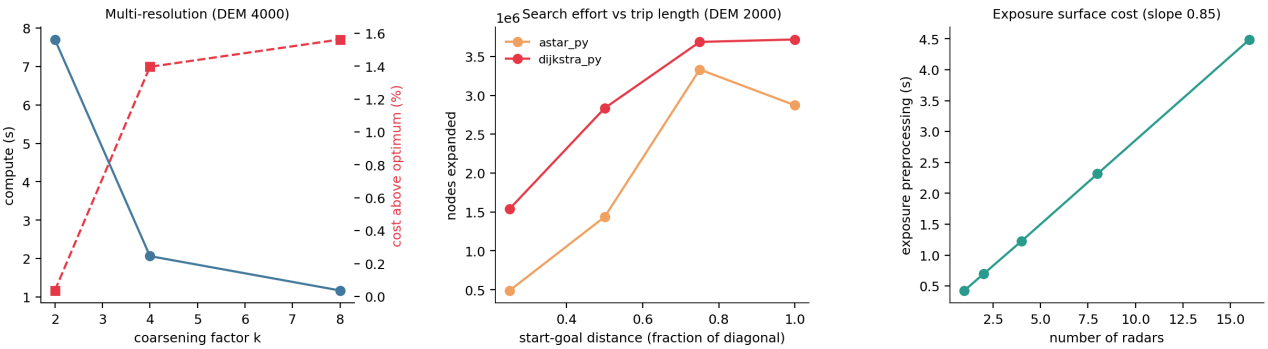


Figure 18. Multi-resolution factor (C3), trip length (C4) and exposure preprocessing cost (C5).

coarsening k	compute	nodes expanded	cost above optimum
2	7.70 s	2,255,759	0.04%
4	2.06 s	672,804	1.40%
8	1.17 s	390,980	1.56%

Table 10. Multi-resolution on the 16 M-node graph (C3); exact SciPy Dijkstra takes 8.98 s and 6 GB.

Trip length (C4). A* pays off on short trips (490,181 vs 1,540,876 nodes at a quarter of the diagonal) and much less on long ones. **Exposure preprocessing (C5)** is linear in the number of radars, about 0.28 s per radar on the 16 M-cell DEM, and is done once per scenario.

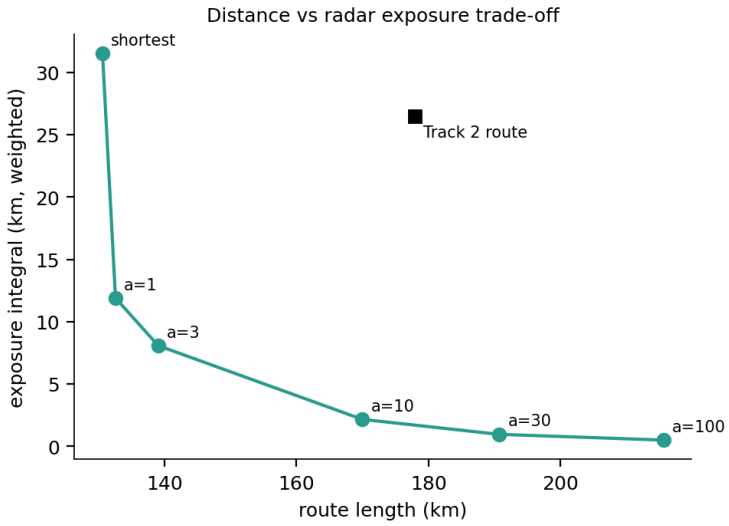


Figure 19. Scenario trade-off between distance and exposure.

9. Part B. Visualisation performance and cross-track comparison

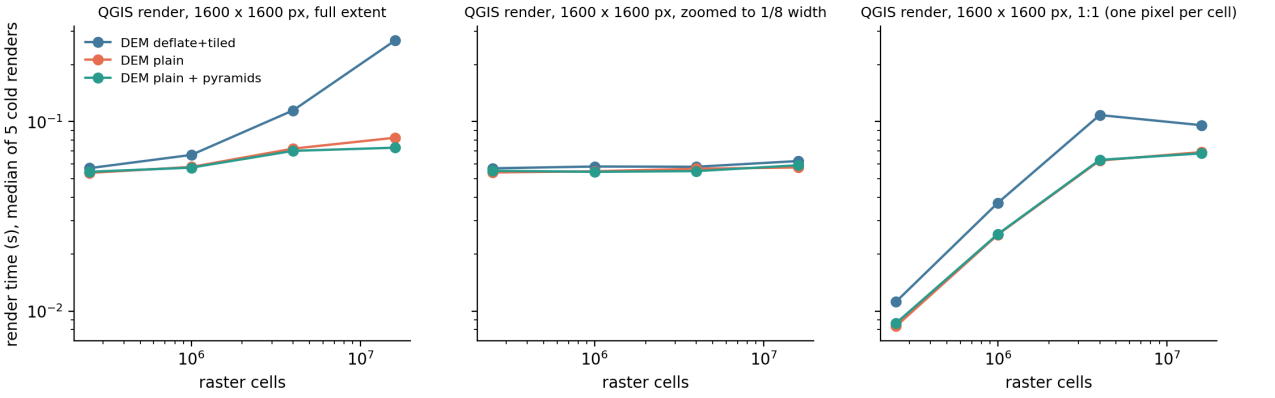


Figure 20. QGIS render time (1600 x 1600 px, median of 5 cold renders) for the DEM stored three ways.

Rendering. QGIS draws plain or pyramided rasters in 55-80 ms at every size: the cost follows the screen (1600 x 1600 px), because GDAL reads only the rows or overview level it needs. Compressed, tiled GeoTIFFs without pyramids are the exception at full extent (0.06 s at 0.25 M cells, 0.27 s at 16 M), since every tile must be decompressed; with GDAL's cache starved to 1 MB the same render took 19.9 s. Each render opens a fresh layer, so GDAL's block cache is cold; the operating-system file cache stays warm. Recommendation: build pyramids for compressed outputs.

Track	unit of work	cost grows with	16 M cells	best lever
1 Radar	one viewshed	cells in range (~range^2)	0.10 s, 242 MB	read only the range window
2 Flight	one viewshed per position	positions x radius^2	18.8 s (97 positions)	keep DEM in memory; fewer positions

3 Corridor	graph search on exposure	nodes (N log N)	9-70 s, 0.7-6 GB	C solver or multi-resolution

Table 11. Cross-track comparison of what drives the cost. Raw runtimes across tracks are not a fair contest because the tracks do different work.

Limitations. Bigger DEMs are also more rugged (they reach the high Himalaya), which mixes size and terrain effects. Peak memory is the process peak working set on Windows; parallel runs report parent and workers separately. The Copernicus model is a surface model (trees and buildings included). The exposure weight (1 - d/range) is a simple stand-in for the radar equation, and the aircraft is a point at a fixed height above ground.